

Panduan Integrasi SSO manAkses Unila

Audiens: Developer aplikasi eksternal (framework bebas — PHP/Laravel, Node.js, Go, mobile Flutter/Android, dll.) yang ingin mengintegrasikan sistem login aplikasinya dengan SSO Universitas Lampung.

Versi dokumen: 1.0 — 2026-04-21 **Scope:** SSO Login (CAS redirect flow) + Logout. Endpoint lain di luar scope dokumen ini.

1. Ringkasan Eksekutif

SSO manAkses Unila adalah sistem Single Sign-On yang menggabungkan **CAS** (Central Authentication Service Unila di `login.unila.ac.id`) dengan penerbitan **JWT** (JSON Web Token, HS256) sebagai tiket identitas untuk aplikasi client.

Alur singkat integrasi:

1. Aplikasi client menyiapkan endpoint `GET /login/sso` yang men-redirect browser ke endpoint SSO manAkses dengan parameter `app_key`.
2. manAkses menangani autentikasi via CAS. Setelah user login, manAkses meng-redirect browser balik ke callback URL aplikasi client dengan JWT di query string.
3. Aplikasi client menerima token di `GET /auth/sso/callback?token=...`, melakukan decode & verifikasi JWT secara lokal (HS256 + `JWT_SECRET` yang di-share dengan manAkses), kemudian membuat session lokal untuk user.

Endpoint production:

- SSO Entry: `https://akses.unila.ac.id/api/live/v1/auth/login/sso?app_key=<APP_KEY>`
 - Callback Client (konvensi wajib): `https://<aplikasi-anda>.unila.ac.id/auth/sso/callback`
-

2. Prasyarat — Checklist ke Admin Manajemen Akses

Sebelum mulai coding, mintalah ke admin Manajemen Akses Unila hal-hal berikut:

#	Kebutuhan	Keterangan
1	APP_KEY (plaintext)	Identifier aplikasi Anda di tabel <code>man_akses.aplikasi</code> . Disimpan di <code>.env</code> client sebagai <code>APP_KEY=...</code> . Kirim via channel aman (bukan email terbuka).
2	Pendaftaran URL aplikasi	URL aplikasi Anda (mis. <code>https://aplikasi-anda.unila.ac.id</code>) didaftarkan di kolom <code>man_akses.aplikasi.url</code> . Ini URL yang dipakai manAkses untuk redirect callback.
3	JWT_SECRET (shared secret)	Kunci HMAC-SHA256 yang dipakai server manAkses untuk sign token. Harus identik dengan value di <code>.env</code> client. Minta via channel aman.
4	Mapping user & role	Admin men-assign user ke aplikasi Anda di <code>man_akses.pj_aplikasi</code> dan <code>man_akses.role_pengguna</code> . User yang tidak di-assign tidak akan bisa login ke aplikasi Anda.
5	Base URL SSO (opsional)	Default production: <code>https://akses.unila.ac.id</code> . Staging/dev bisa berbeda — konfirmasi ke admin.

Catatan penting tentang callback URL: manAkses akan redirect ke `{aplikasi.url}/auth/sso/callback?token=<jwt>`. Path `/auth/sso/callback` adalah **konvensi wajib** yang di-hardcode di server. Pastikan aplikasi client Anda mendaftarkan handler di path ini — tidak bisa diubah ke path lain tanpa modifikasi server manAkses.

3. Scope Dokumen

#	Flow	Endpoint / Route
1	SSO Login — redirect ke CAS, terima callback token	<code>GET {SSO_BASE}/api/live/v1/auth/login/sso?app_key={APP_KEY}</code> → callback <code>{CLIENT_URL}/auth/sso/callback?token=<jwt></code>
2	Logout	Local only — hapus session di client. Tidak ada endpoint API logout di manAkses.

Endpoint yang **tidak** dibahas di dokumen ini (di luar scope):

- `POST /api/live/v1/auth/login` — login non-SSO (machine-to-machine, tanpa password)
- `POST /api/live/v1/auth/login/ssoLogin` — login dengan password via tabel RADIUS
- `POST /api/live/v1/auth/cek_token` — validasi token ke server (client disarankan decode lokal)

4. Flow Diagram — SSO Login



5. Struktur JWT

5.1 Format Token

Token adalah tiga bagian yang dipisah titik:

```
<base64(header)>.<base64(payload)>.<base64(HMAC-SHA256(header.payload, JWT_SECRET))>
```


5.4 Dua Jebakan Encoding yang Harus Diperhatikan

Jebakan #1 — Base64 standar, bukan base64url. Server manAkses menggunakan `base64_encode()` PHP (RFC 4648 standar: karakter `+`, `/`, `=`). Library JWT mainstream default memakai base64url (karakter `-`, `_`, tanpa padding). Jika Anda pakai library JWT mainstream tanpa konfigurasi khusus, **signature check akan selalu gagal.**

Solusinya: decode/encode signature secara manual pakai base64 standar.

Jebakan #2 — Tanda `+` di signature menjadi space di query string. Saat token ditransmisikan via URL (`?token=...`), karakter `+` pada signature kadang ter-decode menjadi spasi oleh browser/server HTTP. Lakukan replace `space → +` sebelum melakukan signature compare.

Contoh fix di PHP:

```
$signature_provided = str_replace(' ', '+', $signature_provided);
```

6. Implementasi Client per Framework

Setiap sub-section berisi 4 bagian standar:

1. Endpoint trigger login (redirect ke SSO)
2. Endpoint callback (decode token + login lokal)
3. Helper decode JWT
4. Handler logout + konfigurasi `.env`

6.1 PHP / Laravel

`.env`

```
APP_KEY=<app_key_plaintext_dari_admin>
JWT_SECRET=<jwt_secret_dari_admin>
SSO_BASE_URL=https://akses.unila.ac.id
```

`routes/web.php`

```
// Trigger SSO login
Route::get('/login/sso', function () {
    $appKey = env('APP_KEY');
    $base = env('SSO_BASE_URL', 'https://akses.unila.ac.id');
    return redirect($base . '/api/live/v1/auth/login/sso?app_key=' . $appKey);
})->name('auth-sso');

// Callback dari manAkses
Route::get('/auth/sso/callback', [\App\Http\Controllers\Auth\SSOController::class, 'callback']
    ->name('auth-sso-callback');

// Logout (protected oleh auth)
Route::middleware(['web', 'auth'])->group(function () {
    Route::post('/auth/logout', [\App\Http\Controllers\Auth\SSOController::class, 'logout'])
        ->name('auth-logout');
});
```

app/Http/Helpers/TokenSSO.php — helper untuk decode + verify JWT:

```

<?php

if (!function_exists('TokenSSO')) {
    function TokenSSO($jwt)
    {
        $secret      = env('JWT_SECRET', 'secret');
        $tokenParts = explode('.', $jwt);

        if (count($tokenParts) !== 3) {
            return null; // format salah
        }

        $header      = base64_decode($tokenParts[0]);
        $payload      = base64_decode($tokenParts[1]);
        $signature_provided = str_replace(' ', '+', $tokenParts[2]); // fix URL encoding
        $payload_decoded = json_decode($payload);

        if (!is_object($payload_decoded)) {
            return null;
        }

        $required_claims = [
            'id_aplikasi', 'url_aplikasi', 'id_pengguna', 'username',
            'nm_pengguna', 'peran_pengguna', 'id_sdm_pengguna', 'id_pd_pengguna',
            'email', 'token_dibuat', 'token_kadarluwasa', 'asal_domain',
            'ip_address', 'sso'
        ];

        foreach ($required_claims as $req) {
            if (!property_exists($payload_decoded, $req)) {
                return null;
            }
        }

        // Cek expiry
        if (($payload_decoded->token_kadarluwasa - time()) < 0) {
            return null;
        }

        // Verifikasi signature (base64 standar)
        $h_b64 = base64_encode($header);
        $p_b64 = base64_encode($payload);
        $sig    = hash_hmac('SHA256', "$h_b64.$p_b64", $secret, true);
        $sig_b64 = base64_encode($sig);

        if ($sig_b64 !== $signature_provided) {
            return null;
        }

        return $payload_decoded;
    }
}

```

app/Http/Controllers/Auth/SSOController.php

```

<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Session;

class SSOController extends Controller
{
    public function callback(Request $request)
    {
        $token = $request->query('token');
        if (!$token) {
            return redirect()->route('auth-login')->with('error', 'Token tidak ada');
        }

        $decoded = TokenSSO($token);
        if (!$decoded || !isset($decoded->sso) || $decoded->sso != 1) {
            return redirect()->route('auth-login')->with('error', 'Token SSO tidak valid');
        }

        // Lookup user lokal
        $user = User::where('username', $decoded->username)->first();

        if (!$user) {
            // Opsional: auto-register dari claim JWT
            $user = User::create([
                'username' => $decoded->username,
                'email' => $decoded->email,
                'nm_pengguna' => $decoded->nm_pengguna,
                'password' => bcrypt(str()->random(32)), // dummy
                // ... field lain sesuai skema client
            ]);
        }

        Auth::login($user);
        Session::put('sso.id_pengguna', $decoded->id_pengguna);
        Session::put('sso.peran', $decoded->peran_pengguna);

        return redirect()->route('dashboard');
    }

    public function logout(Request $request)
    {
        Auth::logout();
        Session::flush();
        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect()->route('home');
    }
}

```

```
}  
}
```

6.2 Node.js (Express)

.env

```
APP_KEY=<app_key_plaintext_dari_admin>  
JWT_SECRET=<jwt_secret_dari_admin>  
SSO_BASE_URL=https://akses.unila.ac.id  
SESSION_SECRET=<random-string-untuk-session>
```

lib/verifySSOToken.js

```

const crypto = require('crypto');

const REQUIRED_CLAIMS = [
  'id_aplikasi', 'url_aplikasi', 'id_pengguna', 'username',
  'nm_pengguna', 'peran_pengguna', 'id_sdm_pengguna', 'id_pd_pengguna',
  'email', 'token_dibuat', 'token_kadarluwasa', 'asal_domain',
  'ip_address', 'sso'
];

function verifySSOToken(jwt) {
  if (!jwt || typeof jwt !== 'string') return null;

  const parts = jwt.split('.');
  if (parts.length !== 3) return null;

  const headerB64 = parts[0];
  const payloadB64 = parts[1];
  // Fix URL encoding issue: '+' jadi space saat transit via URL
  const sigProvided = parts[2].replace(/ /g, '+');

  let payload;
  try {
    const payloadJson = Buffer.from(payloadB64, 'base64').toString('utf8');
    payload = JSON.parse(payloadJson);
  } catch (e) {
    return null;
  }

  // Cek required claims
  for (const claim of REQUIRED_CLAIMS) {
    if (!(claim in payload)) return null;
  }

  // Cek expiry (detik, Unix timestamp)
  if (payload.token_kadarluwasa - Math.floor(Date.now() / 1000) < 0) {
    return null;
  }

  // Verify signature – HMAC-SHA256, base64 standar
  const secret = process.env.JWT_SECRET || 'secret';
  const sigComputed = crypto
    .createHmac('sha256', secret)
    .update(`${headerB64}.${payloadB64}`)
    .digest('base64'); // STANDARD base64, bukan base64url

  if (sigComputed !== sigProvided) return null;

  return payload;
}

module.exports = { verifySSOToken };

```

app.js (Express)

```

require('dotenv').config();
const express = require('express');
const session = require('express-session');
const { verifySSOToken } = require('./lib/verifySSOToken');

const app = express();
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, secure: true, sameSite: 'lax' },
}));

// Trigger SSO login
app.get('/login/sso', (req, res) => {
  const base = process.env.SSO_BASE_URL || 'https://akses.unila.ac.id';
  const url = `${base}/api/live/v1/auth/login/sso?app_key=${encodeURIComponent(process.env.APP_KEY)}`;
  res.redirect(url);
});

// Callback dari manAkses
app.get('/auth/sso/callback', async (req, res) => {
  const token = req.query.token;
  const decoded = verifySSOToken(token);

  if (!decoded || decoded.sso !== true) {
    return res.redirect('/auth/login?error=invalid_sso');
  }

  // Lookup user lokal di DB aplikasi Anda
  // const user = await db.users.findOne({ username: decoded.username });
  // if (!user) { /* auto-register atau redirect ke registration */ }

  req.session.user = {
    id_pengguna: decoded.id_pengguna,
    username: decoded.username,
    nm_pengguna: decoded.nm_pengguna,
    email: decoded.email,
    peran: decoded.peran_pengguna,
  };

  res.redirect('/dashboard');
});

// Logout
app.post('/auth/logout', (req, res) => {
  req.session.destroy(() => {
    res.clearCookie('connect.sid');
    res.redirect('/');
  });
});

app.listen(3000);

```

6.3 Go (Fiber)

`.env`

```
APP_KEY=<app_key_plaintext_dari_admin>  
JWT_SECRET=<jwt_secret_dari_admin>  
SSO_BASE_URL=https://akses.unila.ac.id
```

`internal/sso/verify.go`

```

package sso

import (
    "crypto/hmac"
    "crypto/sha256"
    "crypto/subtle"
    "encoding/base64"
    "encoding/json"
    "errors"
    "strings"
    "time"
)

type Claims struct {
    IDAplikasi      string `json:"id_aplikasi"`
    URLAplikasi     string `json:"url_aplikasi"`
    IDPengguna      string `json:"id_pengguna"`
    Username        string `json:"username"`
    NmPengguna      string `json:"nm_pengguna"`
    PeranPengguna   string `json:"peran_pengguna"`
    IDSMPengguna    string `json:"id_sdm_pengguna"`
    IDPDPengguna    string `json:"id_pd_pengguna"`
    Email           string `json:"email"`
    TokenDibuat     int64  `json:"token_dibuat"`
    TokenKadarluwasa int64  `json:"token_kadarluwasa"`
    AsalDomain      string `json:"asal_domain"`
    IPAddress       string `json:"ip_address"`
    SSO             bool   `json:"sso"`
}

var ErrInvalidToken = errors.New("token SSO tidak valid")

func VerifyToken(jwt, secret string) (*Claims, error) {
    parts := strings.Split(jwt, ".")
    if len(parts) != 3 {
        return nil, ErrInvalidToken
    }
    headerB64 := parts[0]
    payloadB64 := parts[1]
    // Fix URL encoding: '+' jadi space saat transit URL
    sigProvided := strings.ReplaceAll(parts[2], " ", "+")

    payloadBytes, err := base64.StdEncoding.DecodeString(payloadB64)
    if err != nil {
        return nil, ErrInvalidToken
    }

    var c Claims
    if err := json.Unmarshal(payloadBytes, &c); err != nil {
        return nil, ErrInvalidToken
    }

    // Cek claim wajib lewat zero-value check minimal
    if c.IDAplikasi == "" || c.IDPengguna == "" || c.Username == "" ||
        c.TokenKadarluwasa == 0 {

```

```
        return nil, ErrInvalidToken
    }

    // Cek expiry
    if time.Now().Unix() >= c.TokenKadarluwasa {
        return nil, ErrInvalidToken
    }

    // Verify signature HMAC-SHA256, base64 standar
    mac := hmac.New(sha256.New, []byte(secret))
    mac.Write([]byte(headerB64 + "." + payloadB64))
    sigComputed := base64.StdEncoding.EncodeToString(mac.Sum(nil))

    if subtle.ConstantTimeCompare([]byte(sigComputed), []byte(sigProvided)) != 1 {
        return nil, ErrInvalidToken
    }

    return &c, nil
}
```

cmd/main.go (Fiber)

```

package main

import (
    "os"

    "github.com/gofiber/fiber/v2"
    "github.com/gofiber/fiber/v2/middleware/session"
    "example.com/yourapp/internal/sso"
)

func main() {
    app := fiber.New()
    store := session.New()

    // Trigger SSO login
    app.Get("/login/sso", func(c *fiber.Ctx) error {
        base := os.Getenv("SSO_BASE_URL")
        if base == "" {
            base = "https://akses.unila.ac.id"
        }
        url := base + "/api/live/v1/auth/login/sso?app_key=" + os.Getenv("APP_KEY")
        return c.Redirect(url, fiber.StatusFound)
    })

    // Callback
    app.Get("/auth/sso/callback", func(c *fiber.Ctx) error {
        token := c.Query("token")
        claims, err := sso.VerifyToken(token, os.Getenv("JWT_SECRET"))
        if err != nil || !claims.SSO {
            return c.Redirect("/auth/login?error=invalid_sso", fiber.StatusFound)
        }

        sess, _ := store.Get(c)
        sess.Set("id_pengguna", claims.IDPengguna)
        sess.Set("username", claims.Username)
        sess.Set("nm_pengguna", claims.NmPengguna)
        sess.Set("peran", claims.PeranPengguna)
        sess.Save()

        return c.Redirect("/dashboard", fiber.StatusFound)
    })

    // Logout
    app.Post("/auth/logout", func(c *fiber.Ctx) error {
        sess, _ := store.Get(c)
        sess.Destroy()
        return c.Redirect("/", fiber.StatusFound)
    })

    app.Listen(":3000")
}

```

6.4 Mobile (Flutter & Android)

6.4.1 Pertimbangan Khusus Mobile

Flow SSO di mobile **tidak bisa** mengikuti pola server-ke-server biasa karena aplikasi mobile tidak menjalankan web server yang bisa menerima redirect HTTP dari browser. Ada dua opsi implementasi:

Opsi A — In-App WebView + Intercept Redirect (paling umum, direkomendasikan):

1. Aplikasi membuka `WebView` yang load URL SSO manAkses.
2. Pasang listener `onNavigationRequest` / `shouldOverrideUrlLoading`.
3. Ketika WebView mau navigasi ke URL callback (`{url_aplikasi}/auth/sso/callback?token=...`), **intercept** navigasi itu.
4. Extract `token` dari URL, tutup WebView, decode & verify JWT di native code.

Opsi B — App Link / Universal Link (advanced):

- Daftarkan domain callback sebagai App Link (Android) / Universal Link (iOS) yang di-associate dengan aplikasi.
- Browser OS akan langsung membuka aplikasi saat navigasi ke URL callback.
- Butuh konfigurasi `assetlinks.json` di server manAkses dan `Digital Asset Links` — koordinasi dengan admin.

Dokumen ini pakai **Opsi A** (WebView intercept) karena paling sederhana dan tidak butuh perubahan di server SSO.

Catatan keamanan mobile:

- `JWT_SECRET` **jangan** di-hardcode di aplikasi mobile. Ada 2 alternatif:
 1. Verifikasi signature di backend aplikasi sendiri (mobile kirim token ke backend untuk verify) — pola paling aman.
 2. Kalau memang harus verify on-device (mobile-only app tanpa backend), simpan secret di native keystore dan obfuscate — tapi perlu disadari tetap bisa di-extract oleh attacker dengan rooted device.
- Gunakan `flutter_secure_storage` / Android Keystore / iOS Keychain untuk menyimpan JWT di device.

6.4.2 Flutter

`pubspec.yaml`

```
dependencies:  
  flutter:  
    sdk: flutter  
  flutter_inappwebview: ^6.0.0  
  flutter_secure_storage: ^9.0.0  
  crypto: ^3.0.3
```

lib/sso/sso_service.dart — decode & verify JWT

```

import 'dart:convert';
import 'package:crypto/crypto.dart';

class SSOClaims {
  final String idAplikasi;
  final String idPegguna;
  final String username;
  final String nmPegguna;
  final String email;
  final String peranPegguna;
  final String idSdmPegguna;
  final String idPdPegguna;
  final int tokenKadarluwasa;
  final bool sso;

  SSOClaims.fromJson(Map<String, dynamic> j)
    : idAplikasi      = j['id_aplikasi']      as String,
      idPegguna       = j['id_pegguna']       as String,
      username        = j['username']        as String,
      nmPegguna       = j['nm_pegguna']       as String,
      email           = j['email']            as String,
      peranPegguna    = j['peran_pegguna']    as String,
      idSdmPegguna    = (j['id_sdm_pegguna'] ?? '') as String,
      idPdPegguna     = (j['id_pd_pegguna'] ?? '') as String,
      tokenKadarluwasa = j['token_kadarluwasa'] as int,
      sso              = j['sso']              as bool;
}

const _REQUIRED_CLAIMS = [
  'id_aplikasi', 'url_aplikasi', 'id_pegguna', 'username', 'nm_pegguna',
  'peran_pegguna', 'id_sdm_pegguna', 'id_pd_pegguna', 'email',
  'token_dibuat', 'token_kadarluwasa', 'asal_domain', 'ip_address', 'sso',
];

/// Verifikasi JWT SSO manAkses.
/// [secret] hanya perlu kalau verify on-device. Kalau verify di backend,
/// cukup decode (tanpa verify) di mobile, kirim token ke backend.
SSOClaims? verifySSOToken(String jwt, {String? secret}) {
  final parts = jwt.split('.');
  if (parts.length != 3) return null;

  final headerB64 = parts[0];
  final payloadB64 = parts[1];
  // Fix URL encoding: '+' jadi space saat transit URL
  final sigProvided = parts[2].replaceAll(' ', '+');

  Map<String, dynamic> payload;
  try {
    // Base64 STANDAR (bukan base64url)
    final payloadJson = utf8.decode(base64.decode(payloadB64));
    payload = jsonDecode(payloadJson) as Map<String, dynamic>;
  } catch (_) {
    return null;
  }
}

```

```

for (final claim in _REQUIRED_CLAIMS) {
  if (!payload.containsKey(claim)) return null;
}

final nowSec = DateTime.now().millisecondsSinceEpoch ~/ 1000;
if ((payload['token_kadarluwasa'] as int) <= nowSec) return null;

// Verify signature (opsional – hanya kalau secret disediakan)
if (secret != null) {
  final hmacSha256 = Hmac(sha256, utf8.encode(secret));
  final digest = hmacSha256.convert(utf8.encode('$headerB64.$payloadB64'));
  final sigComputed = base64.encode(digest.bytes); // STANDAR base64
  if (sigComputed != sigProvided) return null;
}

return SSOClaims.fromJson(payload);
}

```

`lib/sso/sso_login_page.dart` — halaman login pakai WebView

```

import 'package:flutter/material.dart';
import 'package:flutter_inappwebview/flutter_inappwebview.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'sso_service.dart';

class SSOLoginPage extends StatefulWidget {
  const SSOLoginPage({super.key});
  @override
  State<SSOLoginPage> createState() => _SSOLoginPageState();
}

class _SSOLoginPageState extends State<SSOLoginPage> {
  // Ganti sesuai konfigurasi admin
  static const _ssoBase = 'https://akses.unila.ac.id';
  static const _appKey = 'APP_KEY_DARI_ADMIN';
  static const _callbackHost = 'aplikasi-anda.unila.ac.id'; // dari man_akses.aplikasi.url
  static const _callbackPath = '/auth/sso/callback';

  final _storage = const FlutterSecureStorage();

  @override
  Widget build(BuildContext context) {
    final loginUrl = '$_ssoBase/api/live/v1/auth/login/sso?app_key=$_appKey';

    return Scaffold(
      appBar: AppBar(title: const Text('Login SSO Unila')),
      body: InAppWebView(
        initialUrlRequest: URLRequest(url: WebUri(loginUrl)),
        shouldOverrideUrlLoading: (controller, nav) async {
          final uri = nav.request.url;
          if (uri == null) return NavigationActionPolicy.ALLOW;

          // Intercept navigasi ke callback URL
          if (uri.host == _callbackHost && uri.path == _callbackPath) {
            final token = uri.queryParameters['token'];
            if (token != null) {
              final claims = verifySSOToken(token); // decode only; verify di backend
              if (claims != null && claims.sso) {
                await _storage.write(key: 'sso_token', value: token);
                await _storage.write(key: 'username', value: claims.username);
                if (mounted) Navigator.pushReplacementNamed(context, '/home');
              } else {
                if (mounted) _showError('Token SSO tidak valid');
              }
            }
            return NavigationActionPolicy.CANCEL;
          }

          return NavigationActionPolicy.ALLOW;
        },
      ),
    );
  }

  void _showError(String msg) {

```

```
ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(msg)));
}
}
```

Logout di Flutter:

```
Future<void> ssoLogout() async {
  final storage = const FlutterSecureStorage();
  await storage.deleteAll();

  // Bersihkan cookie WebView supaya next login form CAS fresh
  await CookieManager.instance().deleteAllCookies();

  // Navigasi ke halaman login
}
```

6.4.3 Android Native (Kotlin)

Pola yang sama berlaku untuk Android native — pakai `WebView` + `WebViewClient.shouldOverrideUrlLoading()` untuk intercept callback URL. Contoh Java pada dasarnya identik, cuma sintaksnya berbeda.

`build.gradle` (module app)

```
dependencies {
  implementation "androidx.appcompat:appcompat:1.7.0"
  implementation "androidx.security:security-crypto:1.1.0-alpha06" // EncryptedSharedPreferences
}
```

`SSOTokenVerifier.kt` — decode & verify

```

package com.yourapp.sso

import android.util.Base64
import org.json.JSONObject
import javax.crypto.Mac
import javax.crypto.spec.SecretKeySpec

data class SSOClaims(
    val idAplikasi: String,
    val idPegguna: String,
    val username: String,
    val nmPegguna: String,
    val email: String,
    val peranPegguna: String,
    val idSdmPegguna: String,
    val idPdPegguna: String,
    val tokenKadarluwasa: Long,
    val sso: Boolean
)

object SSOTokenVerifier {

    private val REQUIRED_CLAIMS = listOf(
        "id_aplikasi", "url_aplikasi", "id_pegguna", "username", "nm_pegguna",
        "peran_pegguna", "id_sdm_pegguna", "id_pd_pegguna", "email",
        "token_dibuat", "token_kadarluwasa", "asal_domain", "ip_address", "sso"
    )

    /**
     * Verifikasi JWT SSO manAkses.
     * @param secret kalau null, hanya decode tanpa verify signature
     *             (disarankan verify di backend, bukan di mobile)
     */
    fun verify(jwt: String, secret: String? = null): SSOClaims? {
        val parts = jwt.split(".")
        if (parts.size != 3) return null

        val headerB64 = parts[0]
        val payloadB64 = parts[1]
        // Fix URL encoding: '+' jadi space saat transit URL
        val sigProvided = parts[2].replace(" ", "+")

        val json = try {
            // Base64 STANDAR, BUKAN URL_SAFE
            val payloadBytes = Base64.decode(payloadB64, Base64.DEFAULT)
            JSONObject(String(payloadBytes, Charsets.UTF_8))
        } catch (e: Exception) {
            return null
        }

        // Cek claim wajib
        for (claim in REQUIRED_CLAIMS) {
            if (!json.has(claim)) return null
        }
    }
}

```

```

// Cek expiry
val expiry = json.getLong("token_kadarluwasa")
if (expiry <= System.currentTimeMillis() / 1000) return null

// Verify signature (opsional)
if (secret != null) {
    val mac = Mac.getInstance("HmacSHA256")
    mac.init(SecretKeySpec(secret.toByteArray(Charsets.UTF_8), "HmacSHA256"))
    val sig = mac.doFinal("$headerB64.$payloadB64".toByteArray(Charsets.UTF_8))
    val sigComputed = Base64.encodeToString(sig, Base64.NO_WRAP)
    if (sigComputed != sigProvided) return null
}

return SSOClaims(
    idAplikasi      = json.getString("id_aplikasi"),
    idPengguna      = json.getString("id_pengguna"),
    username        = json.getString("username"),
    nmPengguna      = json.getString("nm_pengguna"),
    email           = json.getString("email"),
    peranPengguna   = json.getString("peran_pengguna"),
    idSdmPengguna   = json.optString("id_sdm_pengguna", ""),
    idPdPengguna    = json.optString("id_pd_pengguna", ""),
    tokenKadarluwasa = expiry,
    sso             = json.getBoolean("sso"),
)
}
}

```

SSOLoginActivity.kt — Activity dengan WebView

```

package com.yourapp.sso

import android.net.Uri
import android.os.Bundle
import android.webkit.WebResourceRequest
import android.webkit.WebView
import android.webkit.WebViewClient
import androidx.appcompat.app.AppCompatActivity

class SSOLoginActivity : AppCompatActivity() {

    companion object {
        const val SSO_BASE      = "https://akses.unila.ac.id"
        const val APP_KEY       = "APP_KEY_DARI_ADMIN"
        const val CALLBACK_HOST = "aplikasi-anda.unila.ac.id"
        const val CALLBACK_PATH = "/auth/sso/callback"
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val webView = WebView(this)
        setContentView(webView)
        webView.settings.javaScriptEnabled = true

        webView.webViewClient = object : WebViewClient() {
            override fun shouldOverrideUrlLoading(
                view: WebView, request: WebResourceRequest
            ): Boolean {
                val uri = request.url
                if (uri.host == CALLBACK_HOST && uri.path == CALLBACK_PATH) {
                    val token = uri.getQueryParameter("token")
                    if (token != null) {
                        val claims = SSOTokenVerifier.verify(token)
                        if (claims != null && claims.sso) {
                            saveTokenSecurely(token, claims.username)
                            finishLogin()
                        } else {
                            showError("Token SSO tidak valid")
                        }
                    }
                }
                return true // cancel navigation
            }
        }
        return false
    }

    val loginUrl = "$SSO_BASE/api/live/v1/auth/login/sso?app_key=$APP_KEY"
    webView.loadUrl(loginUrl)
}

private fun saveTokenSecurely(token: String, username: String) {
    // Pakai EncryptedSharedPreferences (androidx.security.crypto)
    val masterKey = androidx.security.crypto.MasterKey.Builder(this)
        .setKeyScheme(androidx.security.crypto.MasterKey.KeyScheme.AES256_GCM)

```

```

        .build()
    val prefs = androidx.security.crypto.EncryptedSharedPreferences.create(
        this, "sso_prefs", masterKey,
        androidx.security.crypto.EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256,
        androidx.security.crypto.EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256
    )
    prefs.edit()
        .putString("sso_token", token)
        .putString("username", username)
        .apply()
}

private fun finishLogin() {
    setResult(RESULT_OK)
    finish()
}

private fun showError(msg: String) {
    android.widget.Toast.makeText(this, msg, android.widget.Toast.LENGTH_LONG).show()
}
}

```

Logout (Android):

```

fun ssoLogout(context: Context) {
    // Clear EncryptedSharedPreferences
    val prefs = context.getSharedPreferences("sso_prefs", Context.MODE_PRIVATE)
    prefs.edit().clear().apply()

    // Clear WebView cookies supaya next login fresh di CAS
    android.webkit.CookieManager.getInstance().removeAllCookies(null)

    // Navigate ke LoginActivity
}

```

Untuk Java: pola dan method sama persis — cukup port syntax Kotlin ke Java.

`shouldOverrideUrlLoading`, `Uri.getQueryParameter`, `Mac.getInstance("HmacSHA256")`, `Base64.decode(..., Base64.DEFAULT)`, `Base64.encodeToString(..., Base64.NO_WRAP)` semua tersedia langsung di Android SDK.

7. Logout

7.1 Fakta Penting

Tidak ada endpoint API logout di manAkses. Server manAkses tidak menyimpan blacklist token — JWT yang sudah dibuat tetap valid secara signature sampai `token_kadarluwasa` (max 1 jam). Pola standar yang diikuti semua client: **logout dilakukan lokal** di sisi client.

7.2 Langkah Minimum Handler Logout

1. (Opsional — kalau client punya table log_login sendiri) Update log: set `waktu_logout = now()`, `a_sesi_aktif = 0`.
2. Hapus session / clear cookie / delete JWT lokal di client.
3. Redirect user ke halaman landing / form login.

7.3 CAS Single-Logout (Opsional)

Jika user ingin logout juga dari CAS Unila (supaya tidak auto-login di aplikasi lain yang pakai SSO), arahkan browser ke:

```
https://login.unila.ac.id/cas/logout
```

Ini opsional — umumnya tidak dilakukan karena user biasanya hanya ingin logout dari aplikasi yang sedang dipakai. Kalau Anda memilih melakukannya, tambahkan redirect ke URL di atas setelah destroy session lokal.

7.4 Implikasi Keamanan

- JWT yang sudah "di-logout" (dihapus di client) tetap valid secara signature sampai `token_kadarluwasa` tercapai, **jika token sempat bocor ke tempat lain** (log, cache, dll.).
- TTL singkat (1 jam) adalah satu-satunya mitigasi bawaan.
- Jika butuh revoke seketika, Anda perlu implementasi blacklist lokal di client (mis. Redis: `token_hash` → `expired_at`), dan cek di setiap request sebelum trust token.

8. Uji End-to-End (Smoke Test)

1. **Set env:** `APP_KEY` dan `JWT_SECRET` di aplikasi client (sesuai yang dikirim admin).
2. **Jalankan aplikasi client**, buka browser, akses `GET /login/sso`.
3. **Expect:** browser redirect ke `https://akses.unila.ac.id/api/live/v1/auth/login/sso?app_key=...`.
4. **Login di halaman CAS** (`login.unila.ac.id`) dengan kredensial valid.
5. **Expect:** browser redirect balik ke `{CLIENT_URL}/auth/sso/callback?token=<jwt>`.
6. **Inspect token** di `jwt.io` (Decoded panel) — pastikan 14 claim ada, `sso: true`, `token_kadarluwasa` > waktu sekarang. Catatan: **tidak akan bisa verify signature di jwt.io** karena secret HS256 adalah shared antara server & client.
7. **Expect:** setelah callback, aplikasi client login user, redirect ke dashboard.
8. **Uji logout:** POST `/auth/logout` → session lokal ter-clear → redirect ke landing/login.

Uji skenario error:

- `APP_KEY` salah → halaman error dari manAkses.
- User tidak terdaftar di `pj_aplikasi` → callback dengan token yang username-nya tidak ditemukan di DB lokal; handler harus gracefully redirect.
- Token tampered (ubah 1 karakter di payload) → `verify_sso_token` return null, redirect ke error.
- Tunggu > 1 jam, pakai token lama → expired, redirect ke login.

9. Troubleshooting

Gejala	Penyebab yang mungkin	Solusi
Callback tidak pernah sampai ke aplikasi client	URL aplikasi di <code>man_akses.aplikasi.url</code> tidak match dengan URL client	Minta admin update kolom <code>url</code>
Signature verify selalu gagal walau secret sama	Signature di URL: <code>+</code> ter-encode jadi space	Tambahkan <code>replace(' ', '+')</code> sebelum compare (lihat contoh di semua framework)
Signature verify gagal walau sudah replace	Library JWT pakai base64url, server pakai base64 standar	Jangan pakai library JWT mainstream — decode/encode manual (contoh di section 6)
"Required claim missing"	Token bukan dari flow CAS (mis. dari <code>POST /auth/login</code>)	Pastikan link login mengarah ke <code>{SSO_BASE}/api/live/v1/auth/login/sso</code> , bukan endpoint lain
Redirect loop di CAS	<code>app_key</code> tidak terdaftar / expired di <code>man_akses.aplikasi</code>	Minta admin cek row aplikasi & <code>expired_date</code>
Login sukses tapi user 404 di DB lokal	User belum pernah login di aplikasi ini, belum auto-provisioned	Implementasi auto-register di callback (insert ke table user lokal dari claim JWT), atau minta admin assign user di <code>pj_aplikasi</code>
Token expired immediately setelah terima	Clock drift antara server SSO & server client	NTP sync di kedua sisi
Callback bisa sampai, token valid, tapi <code>user.sso == false</code>	Token dari endpoint non-CAS	Validasi <code>payload.sso === true</code> di handler; tolak kalau <code>false</code>

10. Catatan Keamanan

- **HTTPS wajib** di production. Token di query string berisiko ter-log oleh proxy/reverse-proxy/CDN.
- Setelah menerima token di callback, **pindahkan ke cookie** `httpOnly` + `Secure` + `SameSite=Lax` atau server-side session. Jangan biarkan token tersimpan di URL.
- Untuk SPA: `window.history.replaceState({}, '', '/dashboard')` setelah ekstraksi token supaya tidak tersimpan di browser history.
- **Jangan pernah expose `JWT_SECRET` ke frontend/browser.** Semua decode & verify dilakukan di backend client.
- **Validasi `payload.sso === true`** di handler callback. Token `sso: false` datang dari endpoint `POST /auth/login` (non-CAS) dan seharusnya **tidak** dipakai untuk login SSO.
- Pertimbangkan rate limiting di endpoint callback client (mis. `express-rate-limit`, Laravel `throttle`, dll.) untuk mitigasi replay attack.
- Rotasi `JWT_SECRET` periodik — koordinasikan dengan admin manAkses karena client dan server harus sinkron.

11. Appendix

11.1 Glossary

Istilah	Arti
SSO	Single Sign-On — satu kali login, bisa akses banyak aplikasi
CAS	Central Authentication Service — protokol SSO yang dipakai Unila di <code>login.unila.ac.id/cas</code>
JWT	JSON Web Token — format token identitas ter-sign
HS256	Algoritma signing JWT pakai HMAC-SHA256 + shared secret
HMAC	Hash-based Message Authentication Code
Claim	Field di payload JWT (mis. <code>username</code> , <code>email</code>)
<code>app_key</code>	Identifier aplikasi client di <code>man_akses.aplikasi</code> ; dipakai sebagai query param saat trigger SSO
Callback URL	URL di aplikasi client yang dipanggil manAkses setelah user login CAS, format <code>{url_aplikasi}/auth/sso/callback?token=<jwt></code>

11.2 Referensi Endpoint Server manAkses

Komponen	Keterangan
Endpoint SSO	GET <code>https://akses.unila.ac.id/api/live/v1/auth/login/sso?app_key={APP_KEY}</code>
Callback convention	<code>{url_aplikasi}/auth/sso/callback?token=<jwt></code> (path <code>/auth/sso/callback</code> fixed)
Algoritma sign	HS256 (HMAC-SHA256) dengan shared <code>JWT_SECRET</code>
Encoding signature	base64 standar (RFC 4648 dengan <code>+</code> <code>/</code> <code>=</code>), bukan base64url
TTL token	3600 detik (1 jam), tanpa refresh
Required claims	14 claim (lihat Section 5.3)

11.3 Changelog Dokumen

Tanggal	Versi	Perubahan
2026-04-21	1.0	Dokumen awal — SSO Login (CAS) + Logout; contoh kode untuk PHP/Laravel, Node.js, Go, Mobile (Flutter & Android)